PROJECT-BASED LEARNING

PHASE-I EVALUATION

Project Title

Smart Hostel Management & Complaint Resolution System

Team ID: DSCPP-III-2026-T269

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T269
Semester: 3rd
Section: G, Cyber Security, DS 2
Domain: Web Systems & Automation – Data Structures & C++

Team Members

1. Harshit Raj – 2510011339
   Sec: G | rajharshit1807@gmail.com
2. Diviya Negi – 2510010573
   Sec: Cyber Security | diviyanegi51@gmail.com
3. Aditya Verma – 2510380258
   Sec: DS 2 | av0844769@gmail.com

Mentor
Mrs. Ashwini Yeole

Interactions Completed - 2

Problem Statement & Motivation

Traditional hostel management uses manual registers and complaint slips, which can cause delays and make it difficult for students to track their complaints. Serious problems like electrical faults and water leakage may not get immediate attention because complaints are handled on a first-come, first-served basis. This can lead to delays and poor management of urgent complaints.

Motivation

- Manual paper logs lead to lost complaints and zero real-time tracking.
- Critical hazards (short circuits, leaks) delayed without priority queuing.
- Existing systems handle static records rather than live priority triage.

Target Users / Stakeholders

- Primary: Hostel residents (students), wardens, and maintenance staff.
- Secondary: Hostel administration and facility managers.
- Expected benefit: Faster, transparent, priority-driven grievance resolution.

Objectives

Project Objectives

1. To develop a C++ backend system that models students, wardens, staff, and rooms to support organized hostel administration, providing a structured foundation for managing users, rooms, and hostel activities.

2. To implement priority-based complaint sorting and staff task allocation so that urgent hostel issues are handled before routine complaints, helping wardens assign tasks efficiently according to severity.

3. To integrate Data Structures such as Hash Table, Priority Queue, and Queue with C++ OOP concepts including classes, inheritance, polymorphism, and STL for efficient complaint management and practical application of both subjects.

4. To evaluate the system using sample data and real-world edge cases while demonstrating its practical applicability for automated and real-time hostel administration, including server restarts, invalid credentials, and priority reordering.

Key Objective: Automatically triage, prioritize, and assign hostel complaints in real time through a deterministic C-based DSA engine and role-based web architecture.

Proposed Solution

Proposed Approach

1. Student / Staff Data Input & Authentication
2. Automated Priority Score Calculation
3. Hash Table + Priority Queue Matching Logic
4. CSV File Persistence - Store user, complaint, assignment, and activity records in CSV files so data remains available after a server restart.
5. Role Dashboards: Warden & Staff Workflows

Key Features

- Automatic priority scoring based on issue category
- Instant Hash Table lookup by Student ID & Complaint ID
- Priority Queue–based ranking for urgent maintenance
- FIFO Queue buffer for routine cleaning requests
- Action history & undo log powered by Stack & Linked List

Algorithms Used

Algorithm 1 - Priority Handling

1. Read complaint category and severity.
2. Assign a priority level.
3. Insert complaint into Priority Queue.
4. Process highest priority first.

Algorithm 2 - Hash Table Lookup

1. Read Student ID / Complaint ID.
2. Search the Hash Table.
3. Retrieve complaint details.
4. Update the record in CSV.

Algorithm 3 - Dijkstra Shortest Path

1. Model hostel locations as a weighted graph.
2. Set complaint location as the source.
3. Find the shortest route to the staff / service point.
4. Use the route to guide maintenance staff.

Instead of returning a plain static list of complaints, the system computes priority tiers (Critical, High, Medium, Low) and manages active workloads using heap-based priority queues and O(1) hash lookups ? minimizing response time during hostel emergencies.

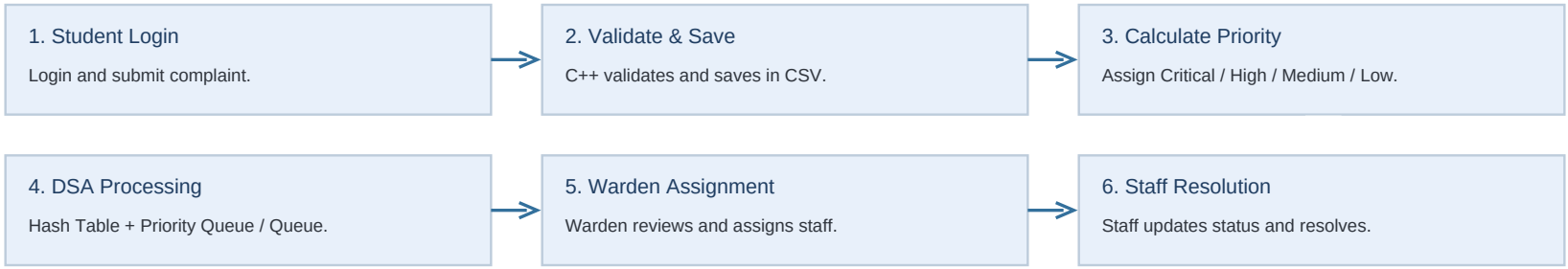
Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures in C	Hash Table, Priority Queue (Heap), Queue, Linked List, Stack	Emergency Triage Engine, Routine Request Buffer, Fast ID Indexing, Activity History & Undo Stack
OOPs with C++	Classes, Inheritance, Polymorphism, STL, File I/O, extern "C" Integration	Role Entities (Student/Warden/Staff), Backend Architecture, CSV Persistence

Requirement: Demonstrate meaningful integration of both subjects in one project.

System Architecture / Workflow

Complaint Handling Workflow



Major Components

- Student Module (login, complaint filing and tracking)
- C++ Backend (authentication, priority calculation and role control)
- DSA Engine (Hash Table, Priority Queue and Queue)
- Warden & Staff Module (assignment, progress and resolution)
- CSV Persistence (users, complaints, assignments and status history)

Data / Control Flow

- Student submits a complaint after login and validation.
- C++ backend calculates a priority score from the issue category.
- Complaint data is stored in CSV for persistence.
- Hash Table provides fast ID lookup; Priority Queue ranks urgent complaints.
- Queue handles routine requests in FIFO order.
- Warden assigns the task; staff updates progress and resolution.
- Updated status and activity history are written back to CSV.

Technology Stack & Methodology

Technology Stack

- Programming: C++17 & Pure C
- Storage: CSV / Text files (in-memory STL)
- Tools / Framework: Standard C++, extern "C"
- IDE: VS Code / Code::Blocks
- Version Control: Git / GitHub

Development Methodology

1. Requirement Analysis
2. System Design
3. Implementation
4. Testing
5. Evaluation

C++ was chosen for direct control over data structures and OOP demonstration; STL containers and CSV file-handling keep the project focused on DS/OOP without extra database overhead.

Initial Progress & Team Contribution

Progress Achieved

- Background / literature study on hostel ticketing systems
- Requirements & role-based workflow identified
- Class structure & DS architecture prepared

Individual Contribution

Member	Role	Contribution
Harshit	Lead	Design, Priority Queue module
Diviya	Developer	Hash Table, complaint matching
Aditya	Testing/Docs	Edge-case testing, docs

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Proposal & Design
2. Phase-II: Development
3. Phase-III: Final Implementation

Expected Outcomes

- Working C++ prototype / system
- Measurable lookup and ranking efficiency
- Practical triage applicability

References

References

[1] Design and Implementation of Automated Secure Web-Based Hostel Management System with Automated Room Allocation and Complaint Tracking, ResearchGate. [Online]. Available:

https://www.researchgate.net/publication/401357924_Design_and_implementation_of_automated_secure_web-based_hostel_management_system_with_automated_room_allocation_and_complaint_tracking

[2] Development of Web-Based Hostel Management System, ResearchGate. [Online]. Available:

https://www.researchgate.net/publication/389336918_Development_of_Web-Based_Hostel_Management_System

[3] GeeksforGeeks, Applications of Priority Queue. [Online]. Available:

https://www.geeksforgeeks.org/dsa/applications-priority-queue/

Thank You

Questions & Suggestions